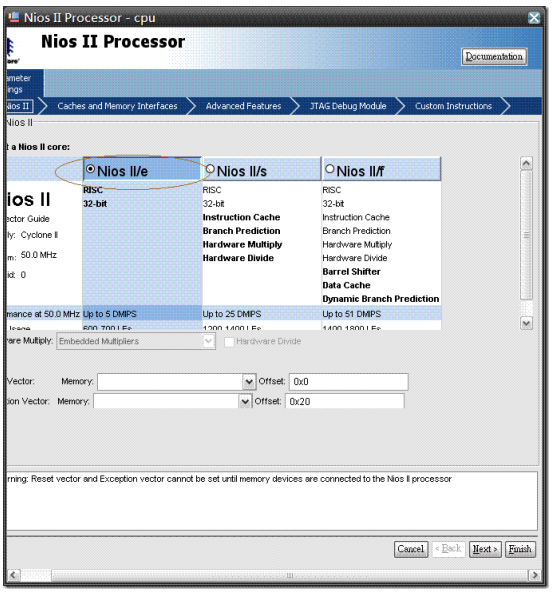
How to create a simple NIOS II System

1) Create a project

   ENSURE THAT PROJECT HAS NO SPACES IN THE PATH NAME. ALSO THE PATH MUST NOT BE TO LONG I RECOMMEND MAKING A FOLDER ON THE ROOT OF THE C DRIVE. THIS WILL CAUSE PROBLEMS LATER FOR DEVELOPING SOFTWARE.

2) Open SOPC builder

3) Select NIOS II Processor , Select NIOS II/e

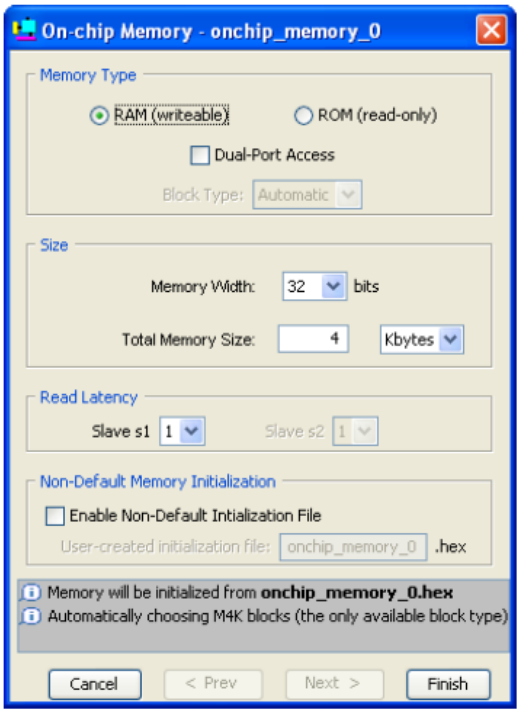


4) Select Memory > On-Chip Memory (RAM or ROM) and click Add

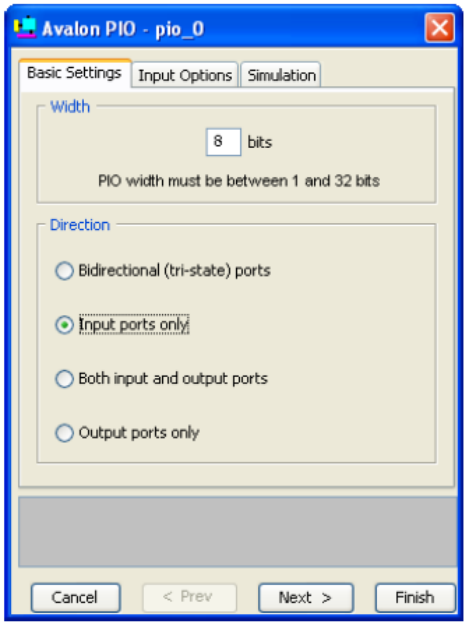
   • In the On-Chip Memory Configuration Wizard window, shown in Figure 8, set the memory width to 32 bits and the total memory size to 4 Kbytes

   • Do not change the other default settings

   • Click Finish, which returns to the System Contents



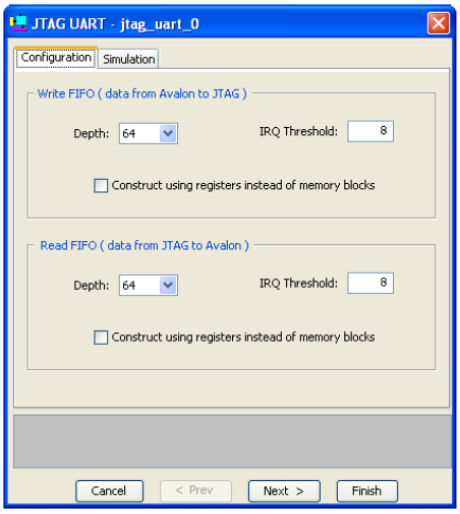
5) Select Components > Other > PIO (Parallel I/O) and click Add to reach the PIO ConfigurationWizard
   - Specify the width of the port to be 8 bits and choose the direction of the port to be Input



6) In the same way, specify the output parallel I/O interface:
   - Select Avalon Components > Other > PIO (Parallel I/O) and click Add to reach the PIO Configuration Wizard again
   - Specify the width of the port to be 8 bits and choose the direction of the port to be Output

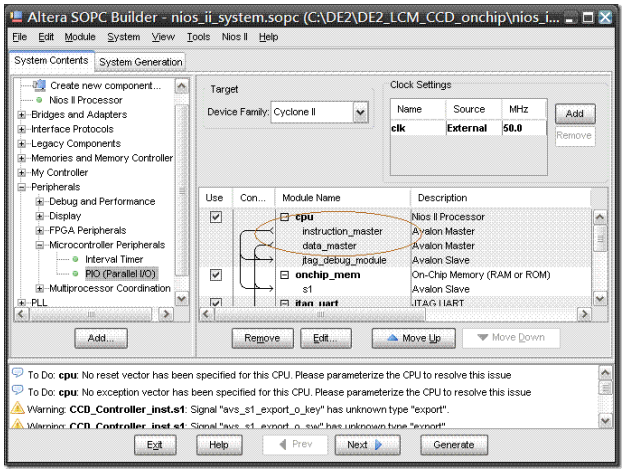
- Click Finish to return to the System Contents tab

7) Select Avalon Components > Communication > JTAG UART and click Add to reach the JTAG UART Configuration Wizard
   - Do not change the default settings
   - Click Finish to return to the System Contents tab

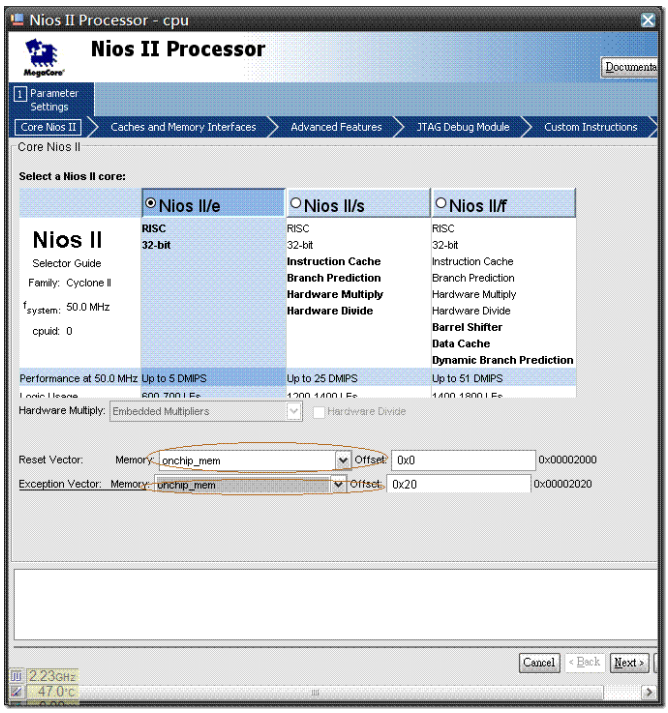


8) The complete system is depicted in Figure 13. Note that the SOPC Builder automatically chooses names for the various components. The names are not necessarily descriptive enough to be easily associated with the target design, but they can be changed. In Figure 2, we use the names Switches and LEDs for the parallel input and output interfaces, respectively. These names can be used in the implemented system. Right-click on the pio_0 name and then select Rename. Change the name to Switches. Similarly, change pio_1 to LEDs

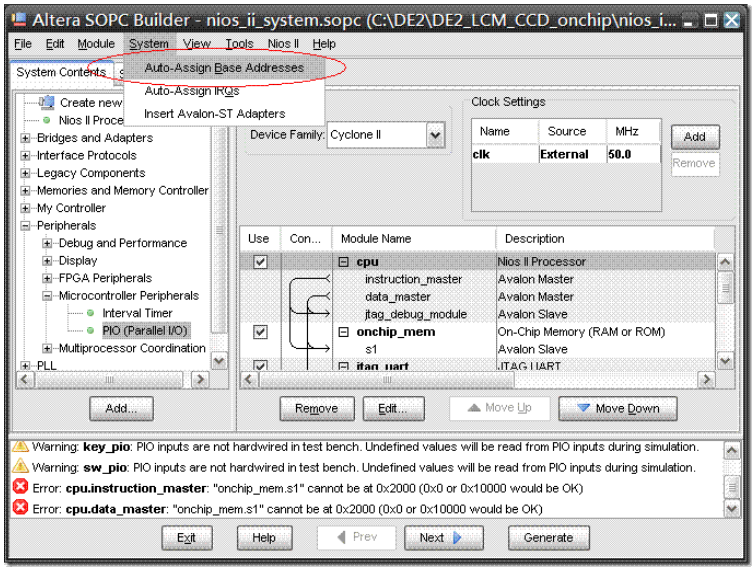
9) Set the reset and exception vector , First double click the CPU



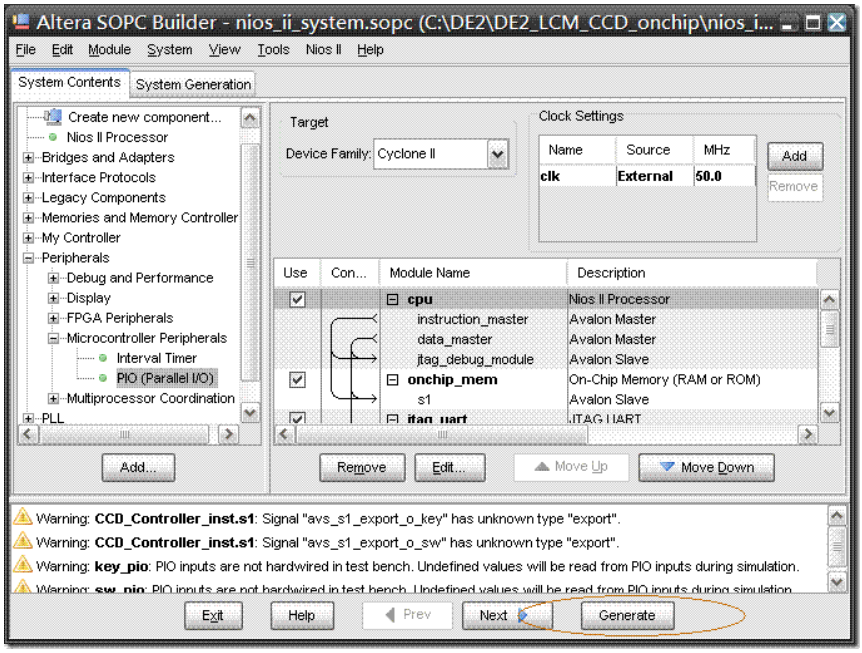
Then select on chip memory in the two drop down boxes



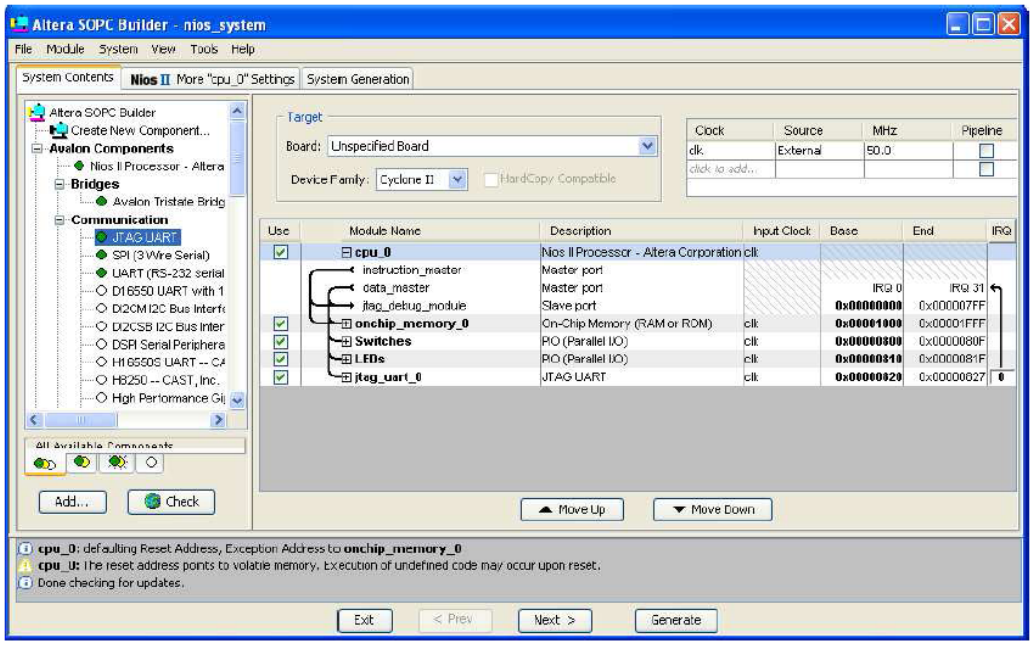
10) Auto assign base addresses , This is down by clicking system then selecting Auto-Assign Base Addresses



Now generate NIOS system

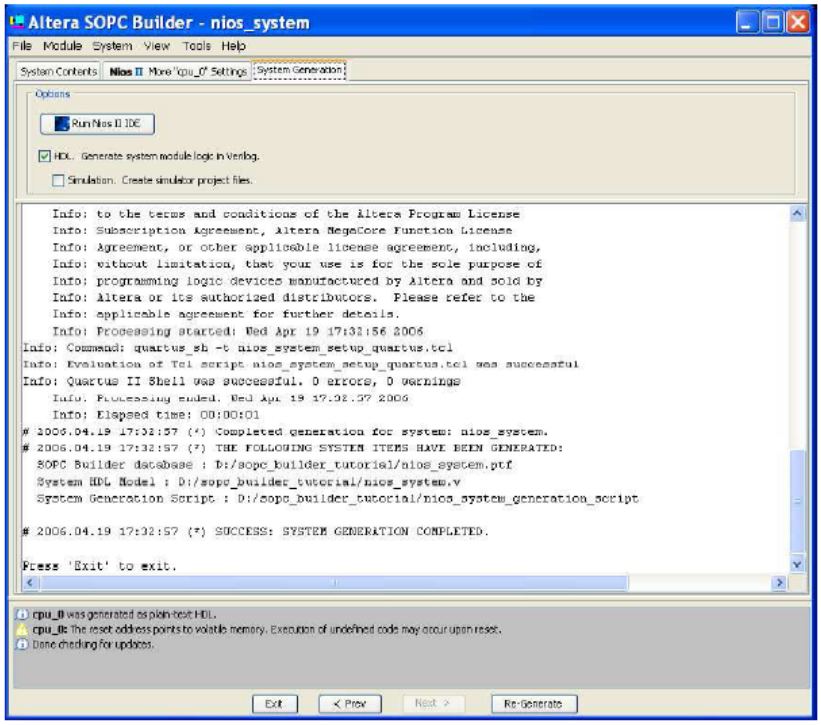


The Complete system should look like the following



11) Having specified all components needed to implement the desired system, it can now be generated. Select the System Generation tab. Turn off Simulation – Create simulator project

files, because in this tutorial we will not deal with the simulation of hardware. Click Generate on the bottom of the SOPC Builder window. The generation process produces the messages displayed in the figure. When the message “SUCCESS: SYSTEMGENERATION COMPLETED" appears, click Exit. This returns to the main Quartus II window.



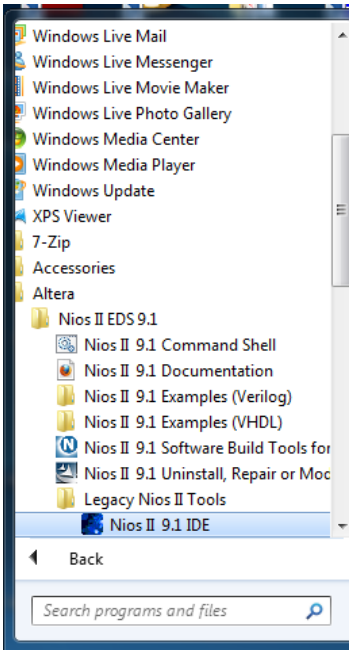
Now That the NIOS II processor has been created the files associated with the NIOS II must be added to the project. **DO NOT ADD ANY OF THE TEST BENCH FILES OR THE NIOS INSTANSITATION THIS WILL CAUSE COMPLIATION ERROR AND HEAD ACHES**

Now create a HDL file and wire NIOS II processor into the top level.

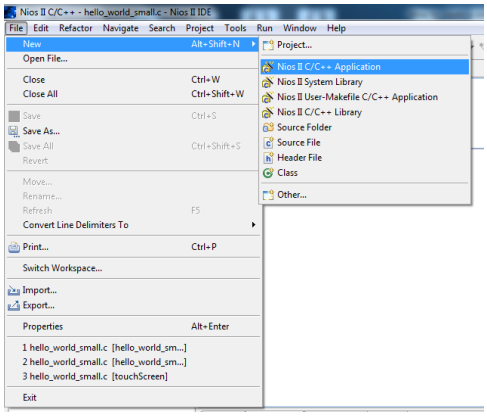
```
nios_ii_system nios_ii (
.clk(CLOCK_50),
.reset_n(KEY[0]),
.out_port_from_the_LEDs(LEDG),
.in_port_to_the_Switches(SW)
);
```

Now compile project code. This completes the hardware section.

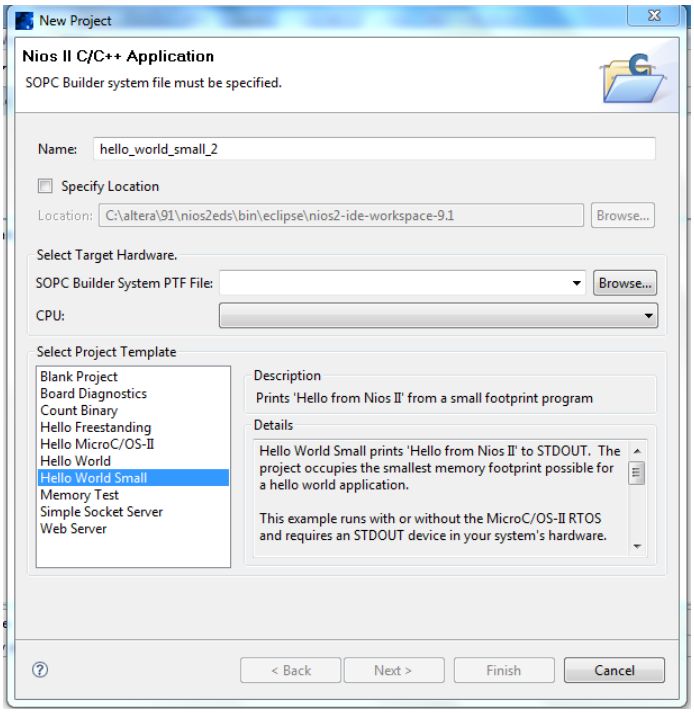
1) Start up the NIOS II Legacy tools



2) Once IDE is open go to file New NIOS II c/C++ Application



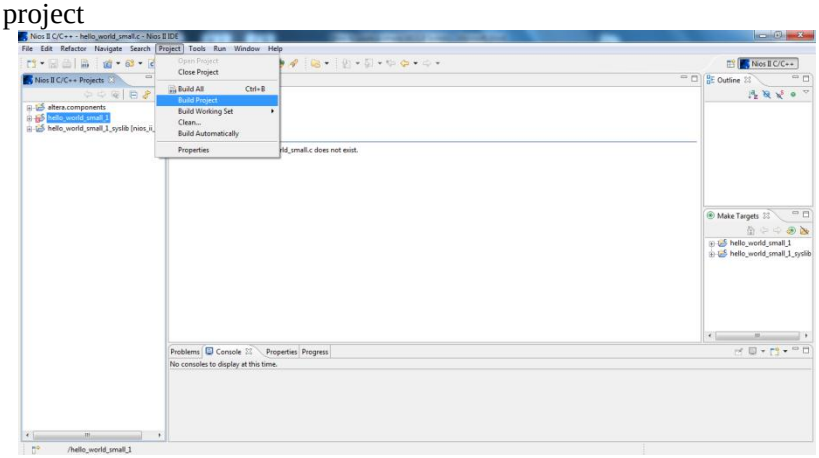
3) Fill out project information. Use Hello World Small project template type to avoid any memory size errors. The SOPC Builder System PFT File or SOPC info file depending on version of Quartus being used can be found in the project directories.



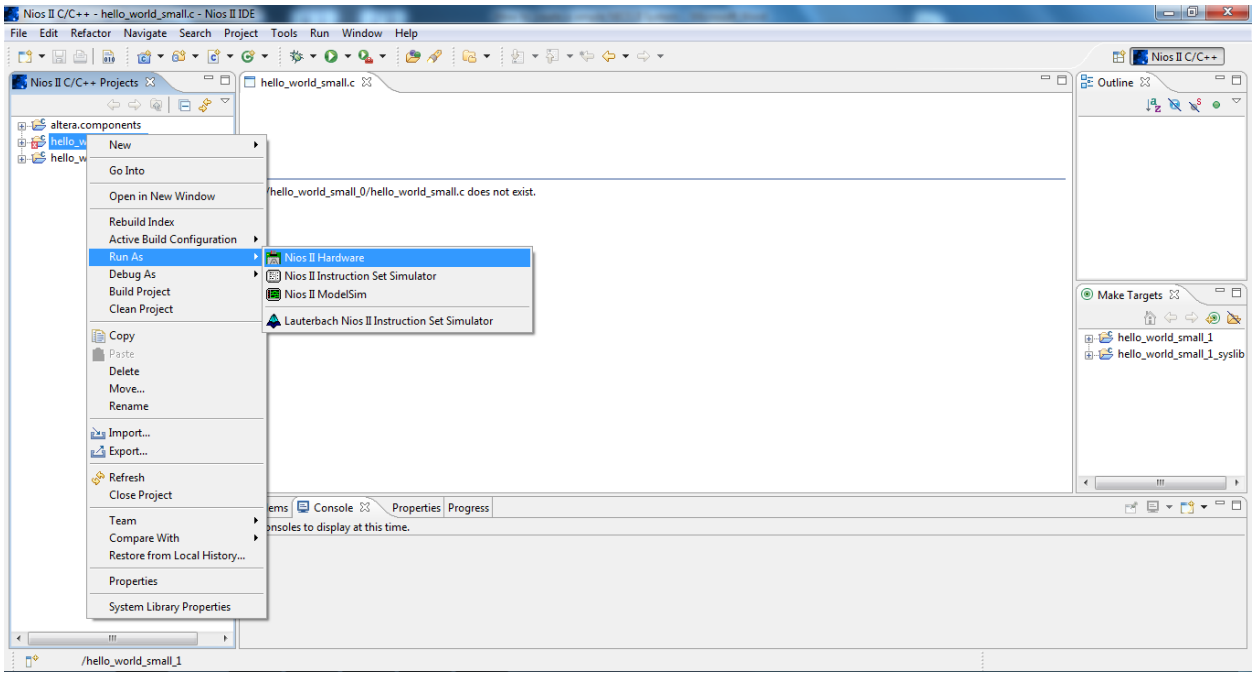
4) Write the following C code

```
#define Switches (volatile char *) 0x0001800
#define LEDs (char *) 0x0001810
void main()
{
        while (1)
        {
                *LEDs = *Switches;
        }
}
```

5) Build the project



6) Upon successful completion of building an ELF file. We can now program the board with the code. This is done by right clicking the project go to run as, and then NIOS II Hardware.



7) Finally verify that the code works by flipping switches and watching LEDS turn on and off. Finally congratulate yourself for making a processor onto an FPGA the first step for creating systems on a chip.